

V-LED 虚拟字符设备驱动系统

项目	内容
课程设计主题	Linux 驱动程序设计
小组	第 5 组
成员	乔思齐（23010342）、曹宇声（23010329）、沈徐鹏（23011850）
实验环境	Ubuntu 24.04.4 LTS / Linux 6.17.0-35-generic / x86_64 / GCC 13.3

摘 要

本课程设计面向 Linux 字符设备驱动的完整工程链路，设计并实现 V-LED 虚拟 LED 显示系统。系统以内核模块 `vled.ko` 为核心，通过 `/dev/vled` 向用户态提供 `open`、`read`、`write`、`poll` 和 `release` 文件操作；以 `PAGE_SIZE` 共享状态页保存完整 JSON，以每次 `open` 独立分配的文件上下文维护读写偏移、稳定快照、已消费版本和写入暂存页；以 `mutex` 保证状态和 JSON 的原子发布，以 `wait queue` 和 `poll` 将周期查询优化为状态变化驱动通知。

用户态由 `vled_cli`、`vled_bridge`、`vled_fd_probe`、`vled_poll_probe`、`vled_verify.sh` 和 `vled_demo.sh` 组成。`bridge` 长期打开设备，首次读取当前状态，随后通过 `poll()` 等待新版本并将通过校验的 UTF-8 JSON 发送至 Windows UDP 9000 端口。Windows 模拟器采用协议解析、UDP 接收、显示模型和 Tk GUI 四层结构，网络线程只产生不可变事件，有界队列将事件交给 Tk 主线程完整替换状态，从而避免跨线程控件访问和部分更新。

项目在 Ubuntu 24.04.4、Linux 6.17.0-35-generic、x86_64、GCC 13.3 环境完成目标验证。P1-P6 证据覆盖零警告构建、设备生命周期、全部业务命令、`PAGE_SIZE` 边界、失败原子回滚、多 FD 独立偏移、稳定快照、4 写者加 4 读者并发、跨机 UDP、Windows GUI、`poll/wait queue`、阻塞与非阻塞读取以及统一入口回归。

关键词：Linux 字符设备；VFS；`PAGE_SIZE`；稳定快照；`wait queue`；`poll`；UDP；Tkinter

Abstract

This course project implements an end-to-end virtual LED system centered on a Linux character-device module. The `/dev/vled` interface exposes `open`, `read`, `write`, `poll` and `release` callbacks. A `PAGE_SIZE` shared state page always contains one complete JSON object, while each open file owns independent read/write offsets, a stable read snapshot, a consumed version and a private staging page. Mutex-protected candidate-state publication provides transactional failure behavior, and a wait queue turns periodic polling into version-driven notification.

The user-space layer contains a command-line client, an event-driven UDP bridge, executable probes, a unified verifier and a reproducible demonstration script. The Windows simulator validates every datagram, transfers immutable events through a bounded queue, and updates Tk widgets only on the main thread. The verified scope includes lifecycle, business commands, boundaries, rollback, concurrent access, cross-machine UDP, GUI behavior, poll semantics and regression evidence on Ubuntu 24.04.4 with Linux 6.17.0-35-generic.

Keywords: Linux character device; VFS; stable snapshot; wait queue; poll; UDP; Tkinter

目 录

摘 要.....	2
Abstract.....	3
目 录.....	4
第 1 章 项目背景、要求与任务组织.....	7
1.1 课程设计背景.....	7
1.2 明示要求与实现映射.....	7
1.3 总体目标与学习目标.....	7
1.4 总体架构与职责边界.....	8
1.5 成员分工与协作接口.....	8
第 2 章 Linux 内核机制理解.....	9
2.1 源码阅读版本与实际运行版本.....	9
2.2 VFS 与 file_operations 调用链.....	9
2.3 dev_t、cdev、class 与 device.....	10
2.4 并发、mutex 与内存可见性.....	10
2.5 wait queue、阻塞读取与 poll.....	10
2.6 内核模块动态装卸.....	11
第 3 章 需求分析与总体设计.....	12
3.1 功能需求.....	12
3.2 非功能、边界与错误需求.....	12
3.3 协议与状态模型.....	12
3.4 核心数据结构分工.....	13
3.5 状态机与事务设计.....	14
3.6 测试与证据设计.....	14
第 4 章 Linux 字符设备驱动实现.....	15
4.1 文件组织与编译方式.....	15
4.2 核心结构体.....	15

4.3 命令解析与版本规则	15
4.4 JSON 转义与 PAGE_SIZE 保证.....	16
4.5 open 与 release	17
4.6 稳定快照 read.....	18
4.7 poll 回调.....	19
4.8 原子 write 与唤醒.....	20
4.9 初始化、失败回退与卸载	21
4.10 不变量与错误处理总结	23
第 5 章 用户态工具、网络桥接与 Windows 模拟器.....	24
5.1 vled_cli.....	24
5.2 vled_bridge 的事件驱动循环.....	24
5.3 自动验收与演示工具	24
5.4 Windows 协议校验.....	24
5.5 网络线程与 Tk 主线程隔离.....	26
5.6 三端数据一致性	26
第 6 章 测试方法、结果与问题分析	27
6.1 测试策略.....	27
6.2 环境与证据链.....	27
6.3 P1: 缓冲区与独立偏移.....	27
6.4 P2: 模拟器.....	28
6.5 P3/P4: 全功能与正式证据	28
6.6 P5/P6: poll 扩展与统一入口	30
6.7 失败—定位—修复链.....	31
6.8 测试真实性与局限.....	31
第 7 章 poll 扩展、优化、安全与局限.....	32
7.1 从周期读取到事件驱动.....	32
7.2 正确性优先的优化原则.....	32
7.3 性能观测解释.....	32

7.4 安全与资源边界.....	32
7.5 工程伦理与适用范围.....	33
第 8 章 团队协作与课程设计日志.....	33
8.1 协作方法.....	33
8.2 可核对时间线.....	33
第 9 章 总结与个人心得.....	34
9.1 项目总结.....	34
9.2 乔思齐个人心得.....	34
9.3 曹宇声个人心得.....	35
9.4 沈徐鹏个人心得.....	35
参考文献.....	37

第 1 章 项目背景、要求与任务组织

1.1 课程设计背景

Linux 设备驱动把硬件或虚拟设备的差异封装在统一文件接口之后。应用程序面对文件描述符调用 `open`、`read`、`write`、`poll` 和 `close`，VFS 根据 `struct file` 关联的 `file_operations` 把请求分派到具体驱动。课程要求选择虚拟字符设备，既能覆盖设备号、`cdev`、`class`、`device`、用户复制和动态装卸，又无需依赖特定外设，便于围绕缓冲区、偏移、并发和错误路径开展可重复实验。

V-LED 把抽象字符设备映射为可观察的 LED 状态：文字、RGB 颜色、亮度、模式和版本。内核只维护可信状态，不承担网络与图形职责；Linux 用户态负责测试和桥接；Windows 负责协议验证与可视化。三端分离使每层都可单独测试，也能通过跨机 UDP 证明整体链路。相关内容见表 1。

1.2 明示要求与实现映射

表 1 课程要求、实现位置与验收映射

要求	实现位置	主要验收
open/read/write/close 回调	driver/vled.c; close 对应 release	T-FOPS、T-CLI、T-CMD
设备号、cdev 与设备节点	alloc_chrdev_region、cdev_add、class/device_create	T-LIFE
用户写入与状态读取	文本命令 + 完整 state JSON	T-CMD、T-JSON
一页大小内核缓冲区	PAGE_SIZE 共享 JSON 页 + per-open 写入页	T-BUF
多进程独立读写偏移	vled_file_context	T-FD、T-READ、T-CON
动态加载卸载	vled_init/vled_exit	装卸循环与内核日志

本报告将“实验明示要求”和“工程扩展”分开论证。PAGE_SIZE、独立偏移和动态装卸属于必须满足的基线；wait queue、poll、事件驱动 bridge、严格 UDP 协议和线程安全 GUI 是在基线稳定后的扩展。

1.3 总体目标与学习目标

- 建立可动态加载、卸载并自动创建设备节点的 Linux 字符设备模块。

- 理解系统调用经 VFS 到 file_operations 的分派，区分 inode、file、cdev、class、device 与 dev_t。
- 把 PAGE_SIZE、per-open 偏移、稳定快照、版本号和错误码写成可执行契约。
- 使用 mutex、wait queue、阻塞/非阻塞 I/O 与 poll 处理共享状态和事件通知。
- 完成 CLI、自动验收、UDP bridge、Windows 模拟器和可复现演示。
- 以原始日志、截图、退出码、提交哈希和 SHA-256 构成证据闭环。相关内容见图 1。

1.4 总体架构与职责边界

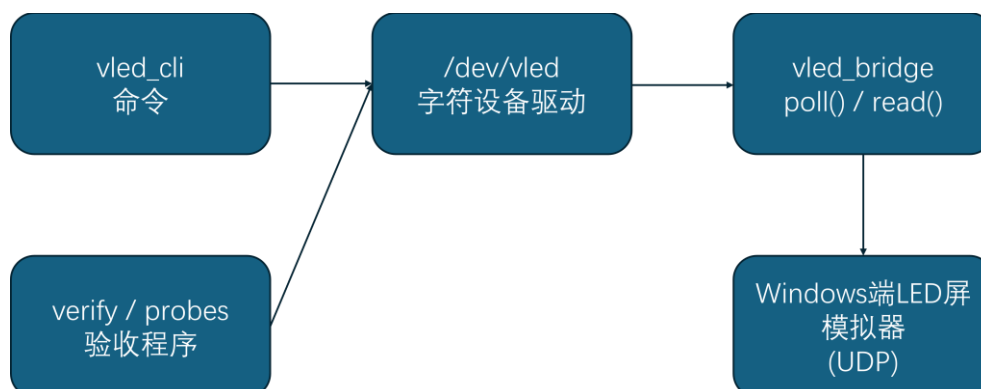


图 1 V-LED 总体架构

内核模块不直接创建 socket，也不直接访问 Windows。这样既遵守内核/用户态职责边界，又能让网络故障不影响字符设备基本功能。bridge 只发送完整且通过字段约束的 state JSON；Windows 端即使收到非法报文，也只记录拒绝原因而不部分更新界面。相关内容见表 2。

1.5 成员分工与协作接口

表 2 团队分工、交付接口与共同工作

成员	主责	交付接口	共同工作
乔思齐 23010342	总体方案、驱动、设备注册、缓冲区、并发与集成	/dev/vled 文本命令与 JSON	协议、联调、验收、报告
曹宇声 23010329	CLI、UDP bridge、用户态测试与演示脚本	命令参数、退出码、UDP 9000	协议、联调、验收、报告
沈徐鹏 23011850	Windows 模拟器、协议解析、UDP 接收与显示	state JSON、线程安全事件	协议、联调、验收、报告

协作以接口契约而非口头约定为中心：驱动输出 canonical JSON，bridge 不修改业务字段，模拟器重新执行独立校验；每次 Linux 验证只针对已经提交并推送的明确哈希；失败证据不覆盖，修复后新建运行记录。该方式使成员可以并行开发，又能在集成时确定责任边界。

第 2 章 Linux 内核机制理解

2.1 源码阅读版本与实际运行版本

课程题目要求概述 Linux 6.18 内核。机制理解部分以 6.18 同系列内核的 VFS、字符设备、等待队列和驱动模型接口为阅读对象；实际构建与运行环境则是 Ubuntu 24.04.4 的 6.17.0-35-generic。前者用于解释通用机制，后者用于给出可复核实验结果。相关内容见图 2。相关内容见表 3。

表 3 内核机制阅读对象与目标环境

维度	阅读/设计对象	实际验证对象
内核	Linux 6.18 机制与同系列文档	6.17.0-35-generic
发行版	不限定	Ubuntu 24.04.4 LTS
架构	通用 VFS/字符设备模型	x86_64
编译器	遵循 Kbuild	GCC 13.3, 匹配内核记录的编译器

2.2 VFS 与 file_operations 调用链

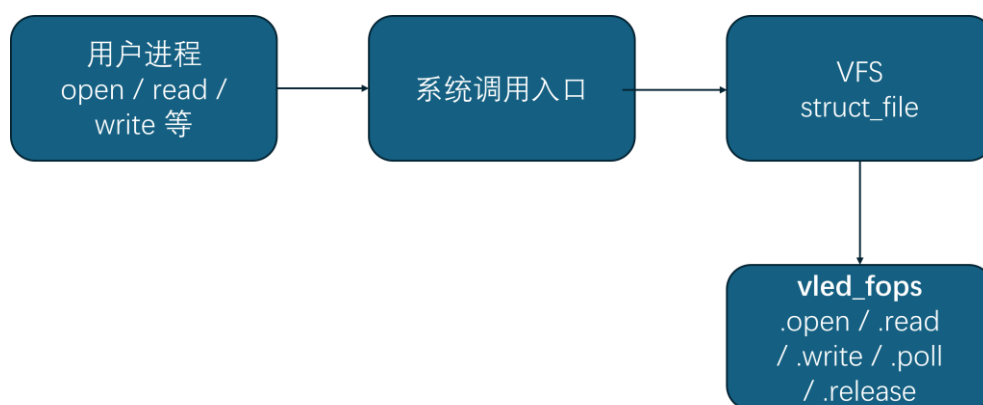


图 2 系统调用经 VFS 到达驱动回调

`inode` 表示文件系统中的对象身份和设备号信息；`struct file` 表示一次打开形成的打开文件描述，可保存访问标志、当前位置和 `private_data`；`cdev` 把字符设备号范围与 `file_operations` 关联。应用 `open /dev/vled` 后，VFS 创建 `struct file` 并调用 `vled_open`，驱动把独立上下文写入 `file->private_data`。`dup` 得到的描述符共享同一个打开文件描述，因此共享同一上下文；两个独立 `open` 才应拥有独立偏移。

read 与 write 的用户地址带有 `__user` 标记，驱动不能直接解引用。V-LED 使用 `copy_from_user` 把本次命令复制到当前 FD 的暂存页，用 `simple_read_from_buffer` 从稳定快照向用户复制。返回值和 `errno` 是接口的一部分：成功返回传输字节数，失败使用负 `errno`，零长度操作返回 0。

2.3 dev_t、cdev、class 与 device

`dev_t` 编码主设备号和次设备号。`alloc_chrdev_region` 动态申请一个编号，避免固定主设备号冲突；`cdev_init` 把 `vled_fops` 绑定到 `cdev`，`cdev_add` 使编号可被 VFS 分派；`class_create` 在 `sysfs` 建立设备类；`device_create` 注册设备对象并让 `udev` 生成 `/dev/vled`。卸载必须以相反顺序执行 `device_destroy`、`class_destroy`、`cdev_del` 和 `unregister_chrdev_region`。

初始化失败路径与正常卸载同样重要。若 `device_create` 失败，应只清理由此前成功获得的资源，不能调用尚未创建对象的销毁函数。代码使用 `goto` 标签形成单调回退链，使每个资源只释放一次。P4 生命周期日志同时检查 `lsmod`、`modinfo`、`stat`、设备节点出现/消失和运行区间 `dmesg`，防止只依据一行“加载成功”。

2.4 并发、mutex 与内存可见性

共享设备状态、`version`、`buffer` 与 `buffer_len` 必须作为一个一致性单元发布。V-LED 在 `vled.lock` 临界区内读取旧状态、验证命令、生成候选状态与候选 JSON，全部成功后一次性替换。读者同样在该锁下捕获完整快照，因此不会看到字段已变而 JSON 未变，或长度与内容不匹配的中间状态。

每文件上下文还有 `ctx->lock`，用于串行化同一个打开文件描述上的读写进度。锁顺序固定为先 `ctx->lock`、后 `vled.lock`，避免不同路径反向加锁形成死锁。阻塞等待前必须释放两把 `mutex`；唤醒后重新检查条件，因为等待队列允许虚假唤醒，状态也可能被其他线程再次改变。

2.5 wait queue、阻塞读取与 poll

等待队列把“条件尚未满足”的进程置于可中断睡眠，状态变化后由 `wake_up_interruptible` 唤醒。`wait_event_interruptible` 每次被唤醒都会重新求值条件；若信号到达则向上返回标准中断错误。`O_NONBLOCK` 文件描述符在没有新版本时不睡眠，而是返回 `EAGAIN`。

`poll` 回调不消费数据，只登记当前 `wait queue` 并报告当前 FD 是否有未消费快照或新版本。有效状态变化使 `version` 递增并唤醒；`STATUS`、重复设置和失败命令不产生事件。这一设计把“可读”定义为相对于当前打开文件描述的版本差，而不是简单地认为共享缓冲区永远非空。相关内容见图 3。

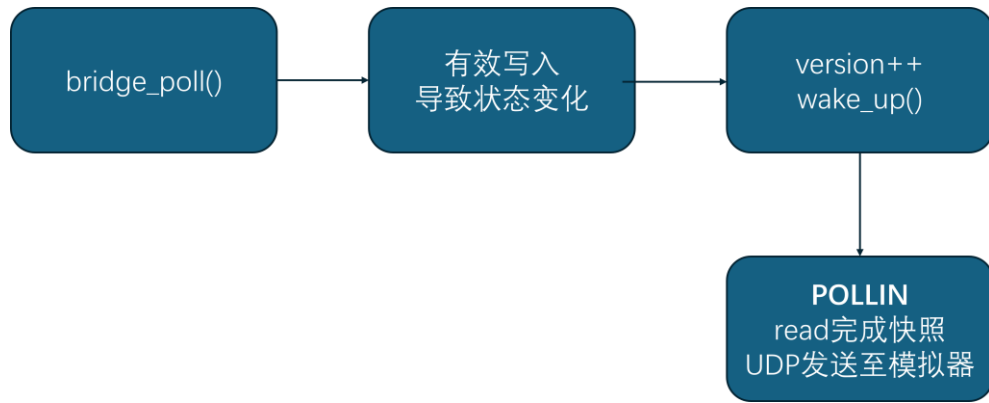


图 3 wait queue 与 poll 事件路径

2.6 内核模块动态装卸

`module_init` 和 `module_exit` 注册初始化与退出口。insmod 把模块装入正在运行的内核，不需要重启；rmmod 在引用允许时执行退出路径。模块元数据声明 GPL 许可证、作者和描述。动态扩展提高实验效率，也提高了失败路径要求：残留设备节点、未释放设备号或错误的销毁顺序都可能污染后续运行。相关内容见表 4。

第 3 章 需求分析与总体设计

3.1 功能需求

表 4 系统需求及其可观察结果

类别	需求	可观察结果
生命周期	构建、加载、自动编号、自动节点、卸载	模块/节点/sysfs/dmesg 一致
命令	TEXT、COLOR、BRIGHTNESS、MODE、CLEAR、STATUS	状态 JSON 与 version 符合规则
读取	完整 JSON、分段读、稳定快照	单一版本、可解析、无拼接
并发	多进程独立 open 与压力访问	偏移独立、无死锁、无无效 JSON
事件	poll、阻塞/非阻塞 read	变化唤醒；无变化超时/EAGAIN
网络与 GUI	UDP 9000、协议校验、可视化	Linux/Windows 字段一致

3.2 非功能、边界与错误需求

非功能要求包括：目标内核零警告构建；失败写入原子回滚；共享状态始终可解析；脚本任一步失败即停止；日志记录命令、退出码、环境和提交；GUI 不跨线程访问 Tk；网络与日志队列有明确上限；卸载后无本轮新增 warning、oops、BUG 或 lockdep 报告。

边界契约把模糊的“一页缓冲区”转化为可测试语义。count 等于 0 返回 0；单次 count 大于等于 PAGE_SIZE 返回 EMSGSIZE；累计写入无法保留结尾 NUL 返回 ENOSPC；TEXT 超过 1023 字节返回 EMSGSIZE；语法、参数或范围错误返回 EINVAL；已识别但不支持的 PIXEL 返回 EOPNOTSUPP；无新版本的非阻塞 read 返回 EAGAIN。相关内容见表 5。

3.3 协议与状态模型

表 5 状态 JSON 字段、类型与含义

字段	类型与范围	含义
type	字符串，固定 "state"	报文类别
width/height	1..128，乘积≤4096	逻辑显示尺寸

字段	类型与范围	含义
text	字符串, UTF-8≤1023 字节	显示文字
color	3 个 0..255 整数	RGB
brightness	0..100 整数	亮度百分比
mode	static/scroll	显示模式
version	非负整数	可观察状态修订号

例子:

```
{"type":"state","width":32,"height":16,"text":"HelloVLED","color":[255,0,0],"brightness":80,"mode":"static","version":12}
```

接口不直接暴露内核 C 结构体, 避免 ABI 对齐、字节序和编译器布局问题。UDP 报文必须是单个完整对象; bridge 和模拟器各自校验, 形成发送端与接收端的双重安全验证。相关内容见图 4。相关内容见表 6。

3.4 核心数据结构分工

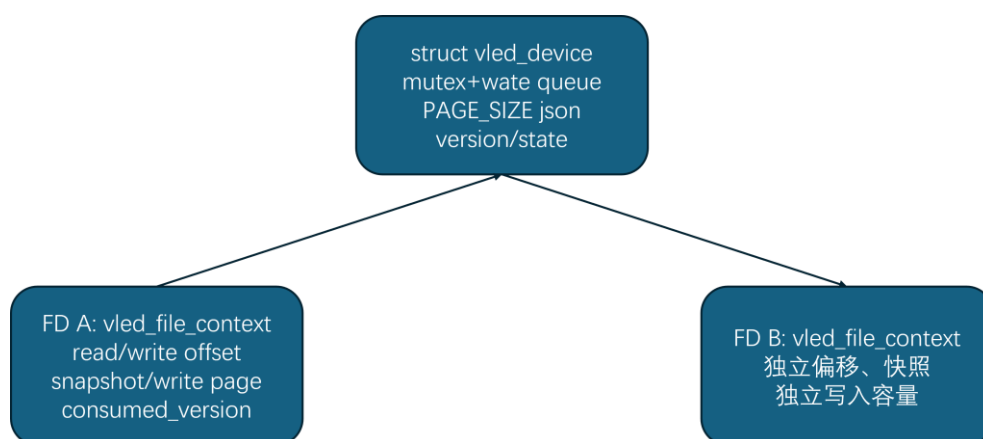


图 4 共享设备状态与 per-open 上下文

表 6 核心数据结构、关键成员与不变量

结构	关键成员	不变量
vled_state	尺寸、text、RGB、brightness、mode、version	只保存已验证可观察状态
vled_device	devno/cdev/class/device、mutex、wait queue、PAGE_SIZE buffer	buffer 与 state/version 同步
vled_file_context	offset、snapshot、consumed_version、write_buffer	每次独立 open 独占; dup 共享

3.5 状态机与事务设计

一次 write 是一条完整命令。解析函数从当前状态复制候选值，只修改候选对象并返回 changed。只有命令合法且候选 JSON 能完整装入 PAGE_SIZE，才提交 state、buffer、buffer_len 和当前 FD 的读刷新标志；否则清除本次暂存片段并保持旧状态。version 仅在 changed 为 true 时递增，因而代表业务状态修订，而不是系统调用计数。相关内容见图 5。

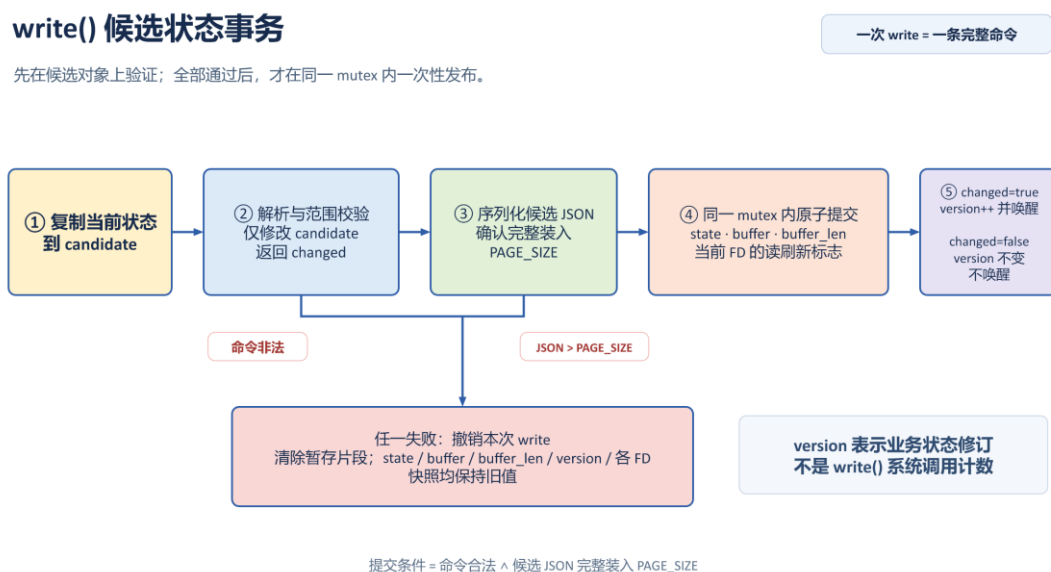


图 5 write() 候选状态事务

3.6 测试与证据设计

验收采用“实现—自动测试—原始证据—文档”四层闭环。测试检查真实系统调用返回值和设备状态，不仅匹配 PASS 字样；并发测试解析每一条采样 JSON；边界测试在失败后再次读取状态、版本和偏移；受控 mutation 临时破坏实现，确认负面测试会失败，随后恢复且不提交 mutation。

第 4 章 Linux 字符设备驱动实现

4.1 文件组织与编译方式

驱动实现位于 `driver/vled.c`，`Makefile` 使用内核 `Kbuild` 外部模块接口。用户态工具与驱动分开构建，避免把用户态头文件或库引入内核。目标运行使用 `/lib/modules/$(uname -r)/build` 指向的匹配 `headers`，并显式采用内核记录的 GCC 13 编译器。

4.2 核心结构体

```

24 struct vled_state {
25     int width;
26     int height;
27     char text[VLED_TEXT_MAX];
28     int red;
29     int green;
30     int blue;
31     int brightness;
32     char mode[VLED_MODE_MAX];
33     unsigned long version;
34 };
35
36 struct vled_device {
37     dev_t devno;
38     struct cdev cdev;
39     struct class *class;
40     struct device *device;
41     struct mutex lock;
42     wait_queue_head_t wait_queue;
43     char buffer[PAGE_SIZE];
44     size_t buffer_len;
45     struct vled_state state;
46 };
47
48 struct vled_file_context {
49     struct mutex lock;
50     loff_t read_offset;
51     size_t write_offset;
52     char *read_snapshot;
53     size_t read_snapshot_len;
54     unsigned long read_snapshot_version;
55     bool read_snapshot_valid;
56     unsigned long consumed_version;
57     bool consumed_version_valid;
58     bool read_refresh_pending;
59     char *write_buffer;

```

代码清单 4-1 `driver/vled.c` — `struct vled_device` / `struct vled_file_context`: 定义共享页、同步对象和 `per-open` 快照与偏移

设备级 `buffer` 明确为 `PAGE_SIZE`，保存最新完整 JSON。`read_snapshot` 和 `write_buffer` 也按页分配到堆，避免把大对象放在有限的内核栈上。

4.3 命令解析与版本规则

```

107 static int vled_parse_command(const struct vled_state *current_state,
108                               char *raw_command, struct vled_state *next,
109                               bool *changed)

```

```

110 {
111     char *command = vled_skip_blanks(raw_command);
112     char *arg;
113     int red;
114     int green;
115     int blue;
116     int brightness;
117     char mode[VLED_MODE_MAX];
118     char tail;
119
120     *next = *current_state;
121     *changed = false;
122
123     if (*command == '\0')
124         return -EINVAL;
125
126     if (vled_has_command(command, "TEXT")) {
127         arg = vled_skip_blanks(command + strlen("TEXT"));
128         if (strlen(arg) >= sizeof(next->text))
129             return -EMSGSIZE;
130
131         if (strcmp(next->text, arg) != 0) {
132             strcpy(next->text, arg, sizeof(next->text));
133             *changed = true;
134         }
135         return 0;
136     }
137
138     if (vled_has_command(command, "COLOR")) {
139         arg = strim(command + strlen("COLOR"));
140         if (sscanf(arg, "%d %d %d%c", &red, &green, &blue, &tail) != 3)
141             return -EINVAL;
142         if (!vled_valid_color(red) || !vled_valid_color(green) ||
143             !vled_valid_color(blue))

```

代码清单 4-2 driver/vled.c — vled_parse_command(): 解析 TEXT、COLOR 等命令并构造候选状态

vled_parse_command 首先复制 current_state，因此任何失败不会污染共享对象。COLOR 使用额外尾字符检测多余参数，三个通道分别验证 0..255；BRIGHTNESS、MODE 采用同样的“解析—参数计数—范围—实际变化”顺序。重复值返回成功但 changed 为 false。CLEAR 只清 text；STATUS 不改变业务字段；PIXEL 明确返回 EOPNOTSUPP。

4.4 JSON 转义与 PAGE_SIZE 保证

```

201 static int vled_json_escape(const char *src, char *dst, size_t dst_size)
202 {
203     size_t in = 0;
204     size_t out = 0;
205     unsigned char ch;
206
207     if (dst_size == 0)
208         return -EMSGSIZE;
209
210     while (src[in] != '\0') {
211         ch = src[in++];
212
213         switch (ch) {
214             case '"':
215             case '\\':
216             case '\n':
217             case '\r':
218             case '\t':
219                 if (out + 2 >= dst_size)
220                     return -EMSGSIZE;
221                 dst[out++] = '\\';
222                 switch (ch) {
223                     case '\n':
224                         dst[out++] = 'n';
225                     case '\r':
226                         dst[out++] = 'r';
227

```



```

228             break;
229             case '\t':
230                 dst[out++] = 't';
231                 break;
232             default:
233                 dst[out++] = ch;
234                 break;
235             }
236             break;
237         default:
238             if (out + 1 >= dst_size)
239                 return -EMSGSIZE;
240             /* Other JSON control bytes are normalized deterministically. */
241             dst[out++] = ch < 0x20 ? ' ' : ch;
242             break;
243     }
244 }
245
246 dst[out] = '\0';
247 return 0;
248 }
249
250 static int vled_build_json(const struct vled_state *state, char *dst,
251                          size_t dst_size, char *escaped_text,
252                          size_t escaped_size)
253 {
254     int ret;
255     int written;
256
257     ret = vled_json_escape(state->text, escaped_text, escaped_size);
258     if (ret)
259         return ret;
260
261     written = snprintf(dst, dst_size,
262                       "{ \"type\": \"state\", \"width\": %d, \"height\": %d, \"text\": \"%s\", \"color\": [%d, %d, %d], \"brightness\": %d, \"mode\": \"%s\", \"version\": %lu\", \"red\": %d, \"green\": %d, \"blue\": %d, \"brightness\": %d, \"mode\": \"%s\", \"version\": %lu}",
263                       state->width, state->height, escaped_text,
264                       state->red, state->green, state->blue,
265                       state->brightness, state->mode, state->version);
266     if (written < 0)
267         return written;
268     if ((size_t)written >= dst_size)
269         return -EMSGSIZE;
270     return written;
271 }
272
273 }
274

```

代码清单 4-3 driver/vled.c — vled_json_escape() / vled_build_json(): 转义文本并生成完整状态 JSON

引号、反斜杠、换行、回车和制表符使用 JSON 转义，其他控制字节被确定性归一为空格。

snprintf 的返回长度必须小于目标页，否则返回 EMSGSIZE，不把截断字符串发布为状态。模块初始化时即构建 version 0 默认 JSON，因此新加载模块第一次读取不是空内容。

4.5 open 与 release

```

275 static int vled_open(struct inode *inode, struct file *file)
276 {
277     struct vled_file_context *ctx;
278     int ret;
279
280     ret = nonseekable_open(inode, file);
281     if (ret)
282         return ret;
283
284     ctx = kzalloc(sizeof(*ctx), GFP_KERNEL);
285     if (!ctx)
286         return -ENOMEM;
287
288     ctx->read_snapshot = kzalloc(PAGE_SIZE, GFP_KERNEL);
289     if (!ctx->read_snapshot) {
290         kfree(ctx);
291         return -ENOMEM;

```

```

292     }
293
294     ctx->write_buffer = kzalloc(PAGE_SIZE, GFP_KERNEL);
295     if (!ctx->write_buffer) {
296         kfree(ctx->read_snapshot);
297         kfree(ctx);
298         return -ENOMEM;
299     }
300
301     mutex_init(&ctx->lock);
302     ctx->read_refresh_pending = true;
303     file->private_data = ctx;
304     return 0;
305 }
306
307 static int vled_release(struct inode *inode, struct file *file)
308 {
309     struct vled_file_context *ctx = file->private_data;
310
311     (void)inode;
312     if (ctx) {
313         kfree(ctx->write_buffer);
314         kfree(ctx->read_snapshot);
315         kfree(ctx);
316     }
317     file->private_data = NULL;
318     return 0;
319 }

```

代码清单 4-4 driver/vled.c — vled_open() / vled_release(): 分配、初始化与释放每文件上下文 nonseekable_open 明确设备不可定位, lseek 返回 ESPIPE。随后依次分配上下文、读取快照页和写入暂存页; 任一分配失败只释放已经获得的对象。release 只释放当前 file->private_data, 不影响共享状态或其他 open。该设计同时满足多进程独立性和 dup 共享打开文件描述的 POSIX 语义。

4.6 稳定快照 read

```

321 static ssize_t vled_read(struct file *file, char __user *user_buf,
322                          size_t count, loff_t *ppos)
323 {
324     struct vled_file_context *ctx = file->private_data;
325     ssize_t ret;
326
327     (void)ppos;
328     if (!ctx)
329         return -EIO;
330     if (count == 0)
331         return 0;
332
333     for (;;) {
334         mutex_lock(&ctx->lock);
335         if (ctx->read_snapshot_valid &&
336             ctx->read_offset < ctx->read_snapshot_len)
337             break;
338
339         mutex_lock(&vled.lock);
340         if (ctx->read_refresh_pending || !ctx->consumed_version_valid ||
341             ctx->consumed_version != vled.state.version) {
342             memcpy(ctx->read_snapshot, vled.buffer, vled.buffer_len);
343             ctx->read_snapshot[vled.buffer_len] = '\0';
344             ctx->read_snapshot_len = vled.buffer_len;
345             ctx->read_snapshot_version = vled.state.version;
346             ctx->read_offset = 0;
347             ctx->read_snapshot_valid = true;
348             ctx->read_refresh_pending = false;
349             mutex_unlock(&vled.lock);
350             break;
351         }
352         mutex_unlock(&vled.lock);
353         mutex_unlock(&ctx->lock);
354
355         if (file->f_flags & O_NONBLOCK)

```

```

356         return -EAGAIN;
357
358         ret = wait_event_interruptible(
359             vled.wait_queue,
360             READ_ONCE(ctx->read_refresh_pending) ||
361             !READ_ONCE(ctx->consumed_version_valid) ||
362             READ_ONCE(ctx->consumed_version) !=
363                 READ_ONCE(vled.state.version));
364         if (ret)
365             return ret;
366     }
367
368     ret = simple_read_from_buffer(user_buf, count, &ctx->read_offset,
369                                   ctx->read_snapshot,
370                                   ctx->read_snapshot_len);
371     if (ret >= 0 && ctx->read_offset >= ctx->read_snapshot_len) {
372         ctx->consumed_version = ctx->read_snapshot_version;
373         ctx->consumed_version_valid = true;
374         ctx->read_snapshot_valid = false;
375     }
376     mutex_unlock(&ctx->lock);
377     return ret;
378 }
379

```

。相关内容见图 6。

代码清单 4-5 driver/vled.c — vled_read(): 维护稳定快照并处理阻塞与非阻塞读取

双 FD 稳定快照时间线

写入发布不会改写已开始的读取；每个 FD 持有自己的稳定快照。

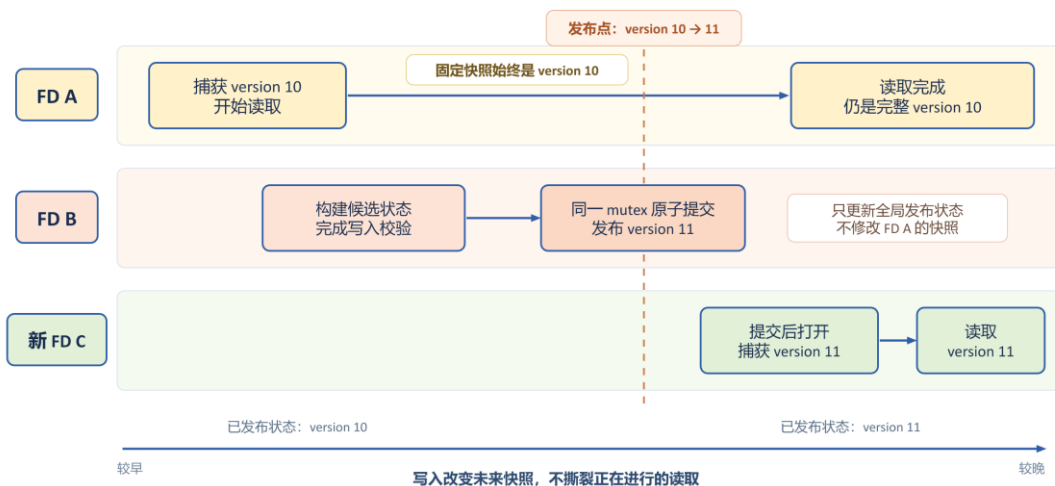


图 6 双 FD 更新期间的稳定快照

读取优先完成当前快照；只有快照无剩余数据时才比较 `consumed_version` 与全局 `version`。捕获动作在设备锁内复制 `buffer` 和长度，之后分段读取只访问私有快照。读完整个快照后记录已消费版本并使快照失效。

4.7 poll 回调

```

380 static __poll_t vled_poll(struct file *file, poll_table *wait)
381 {
382     struct vled_file_context *ctx = file->private_data;
383     __poll_t mask = 0;
384
385     if (!ctx)
386         return EPOLLERR;
387
388     poll_wait(file, &vled.wait_queue, wait);

```

```

389     mutex_lock(&ctx->lock);
390     if (ctx->read_snapshot_valid &&
391         ctx->read_offset < ctx->read_snapshot_len) {
392         mask = EPOLLIN | EPOLLRDNORM;
393     } else {
394         mutex_lock(&vled.lock);
395         if (ctx->read_refresh_pending || !ctx->consumed_version_valid ||
396             ctx->consumed_version != vled.state.version)
397             mask = EPOLLIN | EPOLLRDNORM;
398         mutex_unlock(&vled.lock);
399     }
400     mutex_unlock(&ctx->lock);
401     return mask;
402 }
403

```

代码清单 4-6 driver/vled.c — vled_poll(): 登记等待队列并返回版本就绪事件

poll_wait 把调用者登记到设备等待队列，但函数仍需立即检查当前条件，防止事件在登记前已经发生。若快照仍有剩余字节，或当前 FD 未消费最新 version，则返回 EPOLLIN | EPOLLRDNORM；否则返回 0。就绪状态是每文件上下文相对全局版本的关系。

4.8 原子 write 与唤醒

```

404 static ssize_t vled_write(struct file *file, const char __user *user_buf,
405                          size_t count, loff_t *ppos)
406 {
407     struct vled_file_context *ctx = file->private_data;
408     struct vled_state *candidate;
409     char *escaped_text;
410     char *new_json;
411     char *command;
412     size_t old_offset;
413     bool changed;
414     bool should_wake = false;
415     int json_len;
416     ssize_t ret;
417
418     (void)ppos;
419     if (!ctx)
420         return -EIO;
421     if (count == 0)
422         return 0;
423     if (count >= PAGE_SIZE)
424         return -EMSGSIZE;
425
426     new_json = kzalloc(PAGE_SIZE, GFP_KERNEL);
427     if (!new_json)
428         return -ENOMEM;
429     escaped_text = kzalloc(VLED_ESCAPED_TEXT_MAX, GFP_KERNEL);
430     if (!escaped_text) {
431         kfree(new_json);
432         return -ENOMEM;
433     }
434     candidate = kmalloc(sizeof(*candidate), GFP_KERNEL);
435     if (!candidate) {
436         kfree(escaped_text);
437         kfree(new_json);
438         return -ENOMEM;
439     }
440
441     mutex_lock(&ctx->lock);
442     if (ctx->write_offset >= PAGE_SIZE ||
443         count >= PAGE_SIZE - ctx->write_offset) {
444         ret = -ENOSPC;
445         goto unlock_context;
446     }
447
448     old_offset = ctx->write_offset;
449     if (copy_from_user(ctx->write_buffer + old_offset, user_buf, count)) {
450         memset(ctx->write_buffer + old_offset, 0, count + 1);
451         ret = -EFAULT;

```

```

452         goto unlock_context;
453     }
454     ctx->write_buffer[old_offset + count] = '\0';
455     command = ctx->write_buffer + old_offset;
456     vled_strip_line_end(command);
457
458     mutex_lock(&vled.lock);
459     ret = vled_parse_command(&vled.state, command, candidate, &changed);
460     if (ret)
461         goto rollback_locked;
462
463     if (changed)
464         candidate->version = vled.state.version + 1;
465     json_len = vled_build_json(candidate, new_json, PAGE_SIZE,
466                               escaped_text, VLED_ESCAPED_TEXT_MAX);
467     if (json_len < 0) {
468         ret = json_len;
469         goto rollback_locked;
470     }
471
472     vled.state = *candidate;
473     memset(vled.buffer, 0, sizeof(vled.buffer));
474     memcpy(vled.buffer, new_json, (size_t)json_len);
475     vled.buffer_len = json_len;
476     ctx->write_offset = old_offset + count;
477     ctx->read_offset = 0;
478     ctx->read_snapshot_len = 0;
479     ctx->read_snapshot_version = 0;
480     ctx->read_snapshot_valid = false;
481     ctx->read_refresh_pending = true;
482     should_wake = changed;
483     mutex_unlock(&vled.lock);
484     ret = count;
485     goto unlock_context;
486
487 rollback_locked:
488     memset(ctx->write_buffer + old_offset, 0, count + 1);
489     mutex_unlock(&vled.lock);
490 unlock_context:
491     mutex_unlock(&ctx->lock);
492     kfree(candidate);
493     kfree(escaped_text);
494     kfree(new_json);
495     if (should_wake)
496         wake_up_interruptible(&vled.wait_queue);
497     return ret;
498 }
499

```

代码清单 4-7 driver/vled.c — vled_write(): 检查容量、提交候选状态、失败回滚并按需唤醒

write 先检查 count 和累计容量，再分配候选 JSON、转义缓冲区和候选状态。copy_from_user 只写入当前 FD 的 old_offset 位置；解析或 JSON 构建失败时清除该片段，write_offset 不推进。成功提交后刷新当前 FD 的读取状态，但不重置其他 FD。锁释放、临时对象释放后，只有 changed 为 true 才调用 wake_up_interruptible。

4.9 初始化、失败回退与卸载

```

509 static int __init vled_init(void)
510 {
511     char *escaped_text;
512     int json_len;
513     int ret;
514
515     mutex_init(&vled.lock);
516     init_waitqueue_head(&vled.wait_queue);
517     vled_reset_state();
518
519     escaped_text = kzalloc(VLED_ESCAPED_TEXT_MAX, GFP_KERNEL);
520     if (!escaped_text)
521         return -ENOMEM;

```

```

522     json_len = vled_build_json(&vled.state, vled.buffer,
523                               sizeof(vled.buffer), escaped_text,
524                               VLED_ESCAPED_TEXT_MAX);
525     kfree(escaped_text);
526     if (json_len < 0)
527         return json_len;
528     vled.buffer_len = json_len;
529
530     ret = alloc_chrdev_region(&vled.devno, 0, 1, VLED_DEVICE_NAME);
531     if (ret)
532         return ret;
533
534     cdev_init(&vled.cdev, &vled_fops);
535     vled.cdev.owner = THIS_MODULE;
536
537     ret = cdev_add(&vled.cdev, vled.devno, 1);
538     if (ret)
539         goto unregister_region;
540
541     #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
542         vled.class = class_create(VLED_CLASS_NAME);
543     #else
544         vled.class = class_create(THIS_MODULE, VLED_CLASS_NAME);
545     #endif
546     if (IS_ERR(vled.class)) {
547         ret = PTR_ERR(vled.class);
548         goto del_cdev;
549     }
550
551     vled.device = device_create(vled.class, NULL, vled.devno, NULL,
552                                VLED_DEVICE_NAME);
553     if (IS_ERR(vled.device)) {
554         ret = PTR_ERR(vled.device);
555         goto destroy_class;
556     }
557
558     pr_info("vled: registered /dev/%s major=%d minor=%d\n",
559            VLED_DEVICE_NAME, MAJOR(vled.devno), MINOR(vled.devno));
560     return 0;
561
562 destroy_class:
563     class_destroy(vled.class);
564 del_cdev:
565     cdev_del(&vled.cdev);
566 unregister_region:
567     unregister_chrdev_region(vled.devno, 1);
568     return ret;
569 }
570
571 static void __exit vled_exit(void)
572 {
573     device_destroy(vled.class, vled.devno);
574     class_destroy(vled.class);
575     cdev_del(&vled.cdev);
576     unregister_chrdev_region(vled.devno, 1);
577     pr_info("vled: unregistered /dev/%s\n", VLED_DEVICE_NAME);
578 }
579
580 module_init(vled_init);
581 module_exit(vled_exit);

```

代码清单 4-8 driver/vled.c — vled_init() / vled_exit(): 注册字符设备并按逆序清理资源

初始化顺序为 mutex/wait queue/默认状态、默认 JSON、设备号、cdev、class、device。

class_create 根据内核版本处理参数差异。失败标签严格逆序回收；正常卸载复用相同的逆序原则。

P4 正式证据的三轮装卸以及最终 trap 清理均通过，运行区间只有正常注册和注销日志。相关内容见表 7。

4.10 不变量与错误处理总结

表 7 驱动关键不变量及对应测试

不变量	保持方法	对应测试
共享 JSON 始终完整	候选页构建成功后才提交	T-BUF-07、T-JSON
失败写入无副作用	old_offset + 候选状态 + rollback	T-ROLLBACK
每个 open 独立	private_data 独立页和偏移	T-FD-01..05
快照不混合版本	设备锁下捕获、私有分段读	T-READ-01..05
无意义写入不发布事件	changed 决定 version 与 wake	T-POLL-04

第 5 章 用户态工具、网络桥接与 Windows 模拟器

5.1 vled_cli

vled_cli 提供 read、write 和 loop 三类入口。程序严格检查参数数量、自定义设备路径和整数间隔；write 处理短写，read 循环到完整结果或明确错误；退出码让 shell 和验收脚本可以可靠判断成功与失败。CLI 的目标不是隐藏系统调用，而是以最小封装显式触发 open/read/write/close。

5.2 vled_bridge 的事件驱动循环

bridge 校验 IPv4、端口、设备路径和健康检查超时，创建 UDP socket 后只打开 /dev/vled 一次。初次状态立即读取并发送；之后 poll 等待 POLLIN，读取完整 JSON，通过轻量 canonical state 校验后调用 sendto。超时只用于健康检查，不重复发送旧版本。SIGINT/SIGTERM 修改 sig_atomic_t 标志，使循环退出并关闭 FD 与 socket。

UDP 是无连接协议，sendto 成功只说明本机把报文交给协议栈，不能单独证明 Windows 已收到。相关内容见表 8。

5.3 自动验收与演示工具

表 8 用户态工具职责与失败行为

工具	职责	失败行为
vled_fd_probe	PAGE_SIZE、offset、snapshot、rollback、JSON	任一断言失败退出 1
vled_poll_probe	poll、EAGAIN、信号、高频收敛、指标	逐 ID 报错并非零退出
vled_verify.sh	构建、bridge 黑盒、装卸、P1-P5 回归、dmesg	set -e/trap，尝试清理
vled_demo.sh	跨机文字、颜色、亮度、模式、中文和 CLEAR	任一步失败停止

5.4 Windows 协议校验

vled_protocol.py 对报文长度、UTF-8、JSON 根类型、必填字段、普通整数类型和值域逐项校验。Python 中 bool 是 int 的子类，因此实现用 type(value) is int 排除布尔值。解析成功后返回 frozen dataclass VledState，使接收线程不能在入队后修改事件内容。


```

38     brightness: int
39     mode: str
40     version: int
41
42
43     DEFAULT_STATE = VledState(
44         width=32,
45         height=16,
46         text="",
47         color=(255, 255, 255),
48         brightness=100,
49         mode="static",
50         version=0,
51     )
52
53
54     def _require_plain_int(value: Any, field: str) -> int:
55         if type(value) is not int:
56             raise ProtocolError(f"{field} must be an integer")
57         return value
58
59
60     def validate_state_message(message: Mapping[str, Any]) -> VledState:
61         """Validate one decoded JSON object and return an immutable state."""
62
63         missing = sorted(REQUIRED_FIELDS.difference(message))
64         if missing:
65             raise ProtocolError(f"missing fields: {'', '.join(missing)}")
66         if message["type"] != "state":
67             raise ProtocolError("type must be 'state'")
68
69         width = _require_plain_int(message["width"], "width")
70         height = _require_plain_int(message["height"], "height")
71         if not 1 <= width <= MAX_DIMENSION:
72             raise ProtocolError(f"width must be in 1..{MAX_DIMENSION}")
73         if not 1 <= height <= MAX_DIMENSION:
74             raise ProtocolError(f"height must be in 1..{MAX_DIMENSION}")
75         if width * height > MAX_LED_CELLS:
76             raise ProtocolError(f"width * height must not exceed {MAX_LED_CELLS}")
77
78         text = message["text"]
79         if not isinstance(text, str):
80             raise ProtocolError("text must be a string")
81         if len(text.encode("utf-8")) > MAX_TEXT_BYTES:
82             raise ProtocolError(f"text must not exceed {MAX_TEXT_BYTES} UTF-8 bytes")
83
84         color = message["color"]
85         if not isinstance(color, list) or len(color) != 3:
86             raise ProtocolError("color must be a list of exactly three integers")
87         channels = tuple(_require_plain_int(value, f"color[{index}]")
88                         for index, value in enumerate(color))
89         if any(value < 0 or value > 255 for value in channels):
90             raise ProtocolError("color channels must be in 0..255")
91
92         brightness = _require_plain_int(message["brightness"], "brightness")
93         if not 0 <= brightness <= 100:
94             raise ProtocolError("brightness must be in 0..100")
95
96         mode = message["mode"]
97         if not isinstance(mode, str) or mode not in VALID_MODES:
98             raise ProtocolError("mode must be 'static' or 'scroll'")
99
100         version = _require_plain_int(message["version"], "version")
101         if version < 0:
102             raise ProtocolError("version must be non-negative")
103
104         return VledState(

```

。相关内容见图 7。

代码清单 5-1 simulator/vled_protocol.py — validate_state_message() / parse_datagram(): 校验 UDP state 报文

5.5 网络线程与 Tk 主线程隔离

模拟器后台接收与 Tk 主线程模型

网络线程只生成事件；所有状态变换与界面更新都回到 Tk 主线程执行。

Tk 控件只能由主线程访问

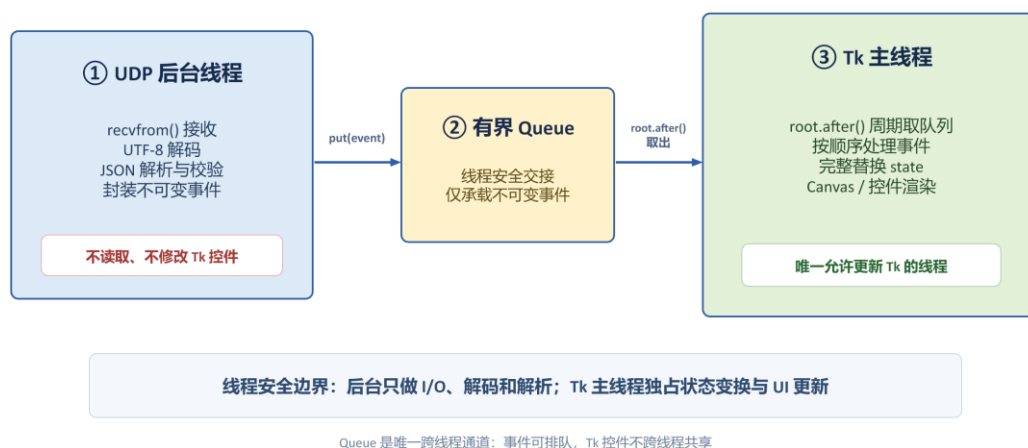


图 7 模拟器后台接收与 Tk 主线程模型

UdpReceiver 不导入 Tkinter。它在非 daemon 线程中使用带超时 socket 接收，解析后写入最大 1024 项的 Queue；队列满时淘汰最旧事件，保证内存有界并优先保留最新状态。Tk 主线程通过 root.after 周期排空队列，完整替换模型，再更新 Canvas、Label 和日志控件。

关闭窗口时先设置 Event、关闭 socket，再 join 接收线程。手动颜色与亮度覆盖只影响 display_state，不覆盖 udp_state；恢复后立即采用最近 UDP 值。本地预览清屏不向驱动发送 CLEAR，避免 GUI 文案制造不存在的控制链路。

5.6 三端数据一致性

驱动负责生成权威 version；bridge 不改字段；模拟器只在整包校验通过后替换状态。Windows 截图显示来源 192.168.57.139、version 1512、#00FFFF、80% 和 static，与 Linux 正式压力测试后 demo 的最终状态一致。version 从 1504 再增加 8 次，是并发压力已产生 1504 个修订后的连续结果，并非异常跳变。

第 6 章 测试方法、结果与问题分析

6.1 测试策略

测试按正常、边界、异常、并发、生命周期、网络和扩展分组。每项包含 Given、When、Then 与防伪检查；脚本不仅输出结论，还检查系统调用返回值、errno、状态 JSON、version、设备节点和内核日志。正式 Linux 只测试 GitHub 上明确提交，不测试 Windows 未提交工作树。相关内容见表 9。

6.2 环境与证据链

表 9 目标 Linux 构建与运行环境

项目	值
发行版	Ubuntu 24.04.4 LTS (Noble Numbat)
内核	6.17.0-35-generic
架构	x86_64
编译器	GCC 13.3.0
Python	Linux 3.12.3; Windows 3.12.9
网络	VMware NAT; Linux 192.168.57.139; Windows VMnet8 192.168.57.1

每个正式运行目录保存环境、构建、功能、边界、多 FD、并发、网络、内核日志、退出码、结果说明和 SHA-256。失败日志单独保留，分析写在 RESULTS.md，不手工改写原始日志。两张截图包含终端命令/退出码与 Windows 状态/UDP 日志，而不是孤立的 PASS 字样。相关内容见表 10。

6.3 P1：缓冲区与独立偏移

表 10 P1 驱动基础语义验证结果

检查	结果	证据
驱动构建	退出码 0，零 warning/error	01-build-final-d8c1c0d.log
PAGE_SIZE/版本/ 零长度	T-BUF-01..06、T-VERSION、T-FOPS-05 PASS	03-probe-final-d8c1c0d.log
原子回滚	T-ROLLBACK PASS	同上

检查	结果	证据
多 FD 与快照	T-FD/T-READ PASS	同上
内核日志	仅正常注册/注销	04-kernel-final-d8c1c0d.log

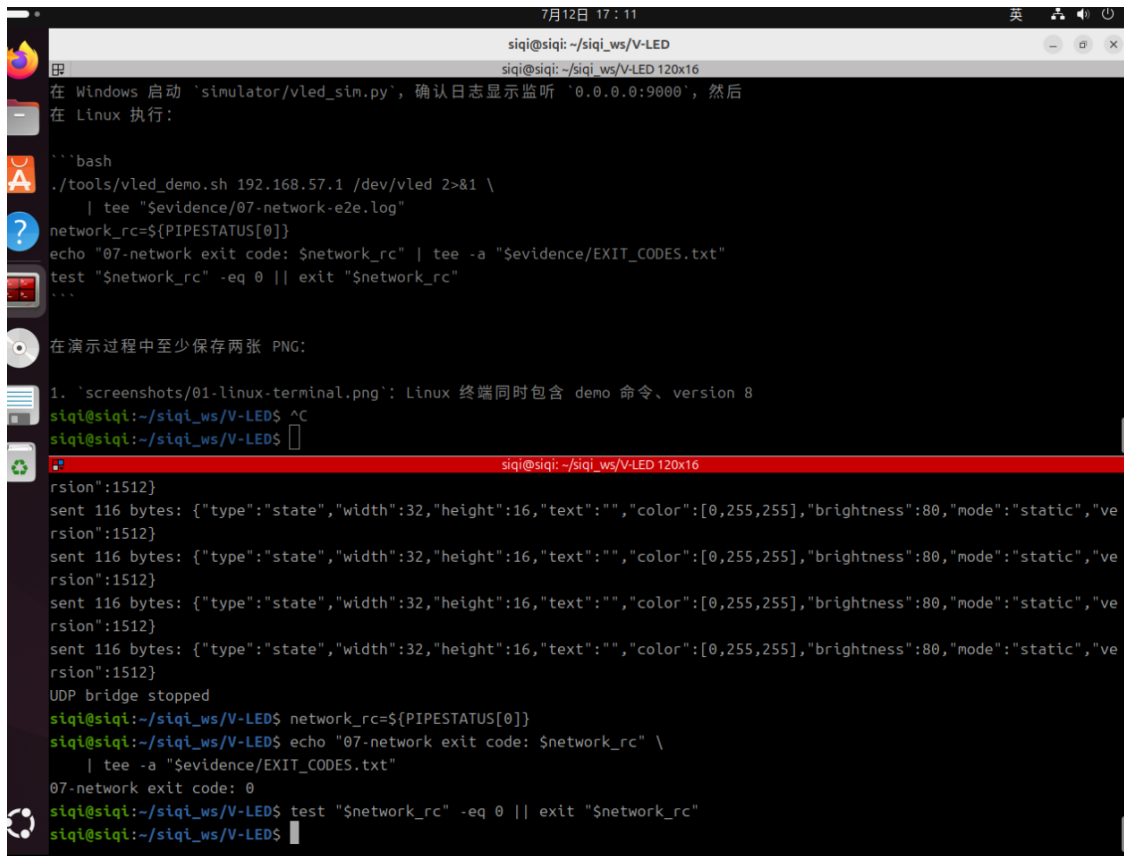
6.4 P2：模拟器

Windows 12 项无 GUI 测试全部通过，并人工覆盖合法 static/scroll、非法 JSON、亮度越界、高频 300 报文、手动覆盖与恢复、本地清屏、200 条日志上限和关闭清理。临时允许 blink 的 mutation 使非法 mode 测试以退出码 1 失败，恢复后重新全过，证明测试能够捕获协议放宽错误。相关内容见图 8。相关内容见图 9。相关内容见表 11。相关内容见表 12。

6.5 P3/P4：全功能与正式证据

表 11 P4 正式证据分组与结果

正式阶段	覆盖	结果
00 环境	OS、内核、headers、Git、网络	exit 0
01 构建	驱动 + tools -Werror	exit 0
02 生命周期	三轮装卸、节点、lsmod/modinfo	exit 0
03 命令	CLI、业务、错误码、version	exit 0
04 边界	PAGE_SIZE、rollback、JSON、ESPIPE	exit 0
05 多 FD	offset、snapshot、dup	exit 0
06 并发	4 writer + 4 reader，各 500 次	exit 0
07 网络	Linux→Windows UDP、信号退出	exit 0
08 内核日志	完整运行区间	无 warning/oops/BUG



```
7月12日 17:11
siqi@siqi: ~/siqi_ws/V-LED
siqi@siqi: ~/siqi_ws/V-LED 120x16
在 Windows 启动 'simulator/vled_sim.py', 确认日志显示监听 '0.0.0.0:9000', 然后
在 Linux 执行:

```bash
./tools/vled_demo.sh 192.168.57.1 /dev/vled 2>&1 \
| tee "$sevidence/07-network-e2e.log"
network_rc=${PIPESTATUS[0]}
echo "07-network exit code: $network_rc" | tee -a "$sevidence/EXIT_CODES.txt"
test "$network_rc" -eq 0 || exit "$network_rc"
```

在演示过程中至少保存两张 PNG:

1. 'screenshots/01-linux-terminal.png': Linux 终端同时包含 demo 命令、version 8

siqi@siqi:~/siqi_ws/V-LED$ ^C
siqi@siqi:~/siqi_ws/V-LED$

rsion":1512}
sent 116 bytes: {"type":"state","width":32,"height":16,"text":"","color":[0,255,255],"brightness":80,"mode":"static","ve
rsion":1512}
sent 116 bytes: {"type":"state","width":32,"height":16,"text":"","color":[0,255,255],"brightness":80,"mode":"static","ve
rsion":1512}
sent 116 bytes: {"type":"state","width":32,"height":16,"text":"","color":[0,255,255],"brightness":80,"mode":"static","ve
rsion":1512}
sent 116 bytes: {"type":"state","width":32,"height":16,"text":"","color":[0,255,255],"brightness":80,"mode":"static","ve
rsion":1512}
UDP bridge stopped
siqi@siqi:~/siqi_ws/V-LED$ network_rc=${PIPESTATUS[0]}
siqi@siqi:~/siqi_ws/V-LED$ echo "07-network exit code: $network_rc" \
| tee -a "$sevidence/EXIT_CODES.txt"
07-network exit code: 0
siqi@siqi:~/siqi_ws/V-LED$ test "$network_rc" -eq 0 || exit "$network_rc"
siqi@siqi:~/siqi_ws/V-LED$
```

图 8 P4 Linux 终端与网络退出码证据

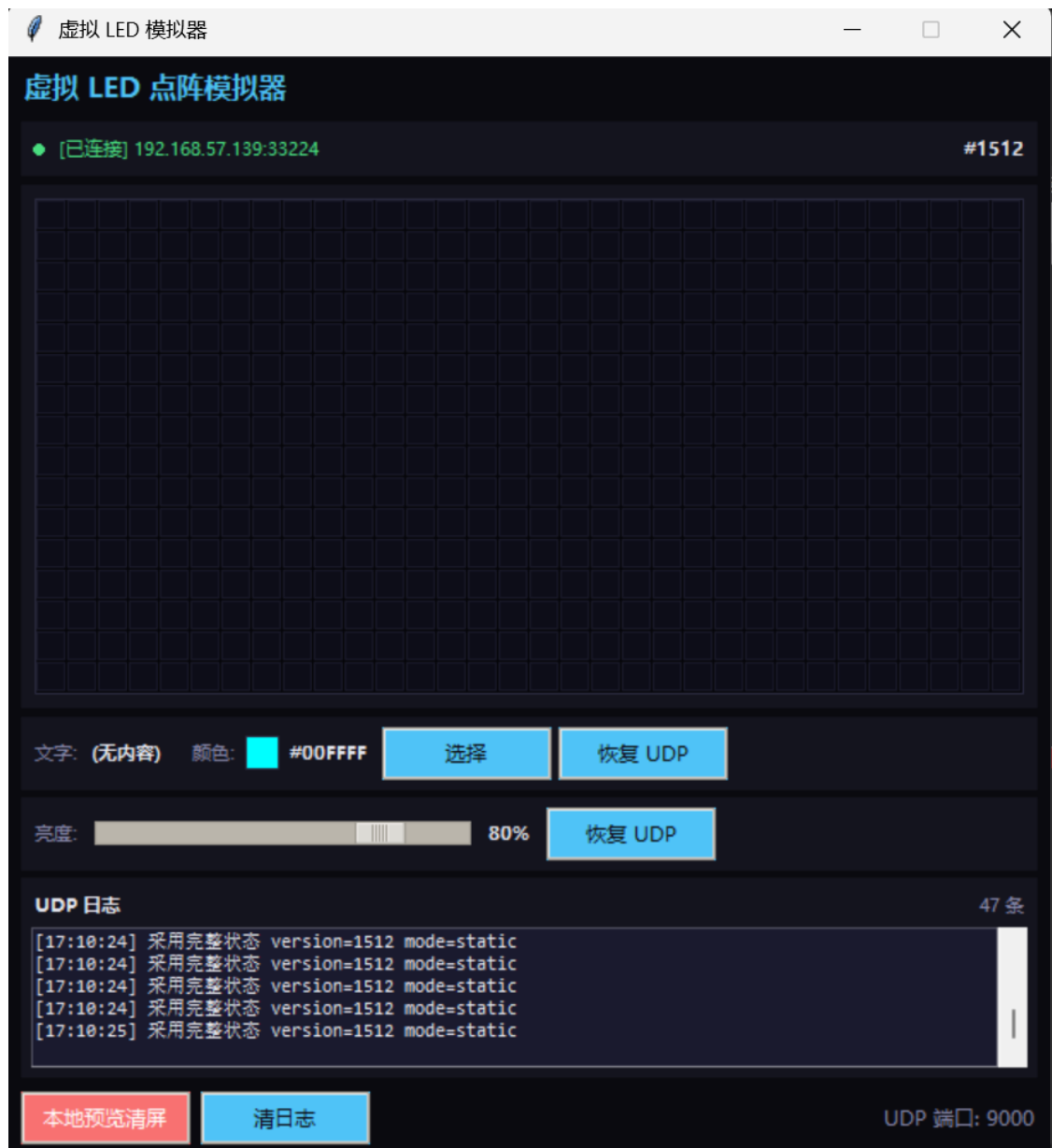


图 9 P4 Windows VLED GUI 与 UDP 日志证据

6.6 P5/P6：poll 扩展与统一入口

表 12 P5 poll 扩展验收结果

| 验收项 | 结果 |
|-----------|--------------------------|
| T-POLL-01 | 初次打开当前状态可读 |
| T-POLL-02 | 已消费版本 poll 超时，不忙等 |
| T-POLL-03 | 实际变化返回 POLLIN/POLLRDNORM |
| T-POLL-04 | STATUS/重复值/非法写入不产生事件 |
| T-POLL-05 | O_NONBLOCK 无新版本返回 EAGAIN |

| 验收项 | 结果 |
|-----------|---|
| T-POLL-06 | 阻塞 read 可被信号中断 |
| T-POLL-07 | 高频写最终状态有效且收敛 |
| T-POLL-08 | 空闲 0.351s、0 CPU ticks、发送 2 次、延迟 0.429ms |

P6 在提交 2ea2346 上再次运行统一入口，驱动、四个 C 工具、bridge 黑盒、P1/P3/P5 回归、三轮生命周期和内核日志均通过。P6 的响应延迟观测为 0.718 ms，与 P5 数值不同，说明该指标受单次调度与环境影响，报告只陈述观测，不把差异解释为算法退化。相关内容见表 13。

6.7 失败—定位—修复链

表 13 实施问题、定位与回归结果

| 阶段 | 现象 | 定位 | 修复与回归 |
|------|---------------------------|----------------------------|-------------------------|
| P1 | current 宏冲突、no_llseek 已移除 | 目标 6.17 编译输出 | 改名参数、移除旧成员；重建 |
| P1 | vled_write 栈帧 1136B 警告 | Kbuild -Wframe-larger-than | 候选状态移到堆；零警告 |
| P3 | 零长度 write 被测试当作空白命令 | 正确驱动返回 0 | 改用三个空格验证 EINVAL |
| P5-1 | 旧 probe 阻塞 | strace 发现 O_RDWR 阻塞读 | 改 O_NONBLOCK 并断言 EAGAIN |
| P5-2 | 旧快照与新版本拼接 | helper 额外 read 跨越边界 | 按剩余长度读取 |
| P5-3 | 并发 reader 拼接下一版本 | 首次完整 JSON 后仍继续读 | 协议校验成功即返回 |

这些失败没有被删除。它们揭示了事件驱动语义对旧测试假设的影响：驱动行为正确后，测试工具本身也必须按长期打开 FD 的版本边界读取。保留失败输出并解释修复，比只展示最终 PASS 更能证明工程过程真实且可复核。

6.8 测试真实性与局限

测试真实性通过受控 mutation、JSON 解析、状态回读、退出码和内核日志交叉验证。仍需承认局限：UDP 不提供传输确认；性能样本次数有限；未启用 lockdep/KASAN 长时压力；设备权限正式方案仍应使用 udev 组规则；Windows 截图只能证明当次界面状态，不能代表所有网络环境。

第 7 章 poll 扩展、优化、安全与局限

7.1 从周期读取到事件驱动

旧方案按固定间隔打开或读取设备，即使状态不变也会唤醒进程并重复发送。它实现简单，但空闲开销、重复报文和响应延迟同时受轮询周期约束。事件驱动方案让 `bridge` 单次打开设备，用 `poll` 等待 `version` 变化；超时只记录健康状态，不发送旧数据。

7.2 正确性优先的优化原则

优化没有改变冻结的 `PAGE_SIZE`、独立偏移、稳定快照、原子回滚和 `version` 语义。`poll` 只是暴露“当前 `FD` 是否有未消费版本”，`read` 仍返回完整快照。`P5` 每次修改后先跑 `P1-P4` 回归，再判断扩展是否可接受，避免用性能改进破坏基础正确性。相关内容见表 14。

7.3 性能观测解释

表 14 P5 事件驱动性能观测

| 指标 | P5 观测 | 解释边界 |
|-----------|----------|----------------|
| 空闲窗口 | 0.351 s | 短窗口，仅验证无忙等 |
| CPU ticks | 0 | 该窗口内未累计可见 tick |
| 发送次数 | 2 | 初始状态 + 一次变化 |
| 响应延迟 | 0.429 ms | 单机目标环境调度观测 |

7.4 安全与资源边界

- 用户指针只通过内核 `copy helper` 访问；长度在复制前检查。
- 命令、`TEXT`、`JSON`、`UDP` 报文、显示尺寸、事件队列和日志均有上限。
- 共享系统不应 `chmod 666`，应使用 `udev` 规则和专用组执行最小权限。
- 内核不直接联网，降低攻击面并避免在内核实现应用层协议。
- 非法报文只产生诊断事件，不修改显示状态；日志不记录凭据。

7.5 工程伦理与适用范围

课程项目应如实区分已验证、未验证和环境观测，不伪造 Linux PASS 或截图；开源仓库不提交密码、Token、私钥和敏感公网地址；引用代码和文档注明来源。V-LED 是教学虚拟设备，不应直接用于安全关键显示、工业控制或医疗提示。若扩展到真实硬件，需要补充电气安全、故障安全、权限审计、协议认证和长期可靠性验证。

第 8 章 团队协作与课程设计日志

8.1 协作方法

团队采用分层接口、最小可审查提交和阶段任务边界。驱动、模拟器、测试、工具、文档与报告分别提交；Linux 只运行已经推送的明确哈希；首次历史对齐后只允许普通 fast-forward 推送。每个阶段先定义验收，再实现，再固化证据，避免“界面能显示”被误当作驱动正确。相关内容见表 15。

8.2 可核对时间线

表 15 课程设计日志

| 时间 | 提交/证据 | 成果 | 自我评价 |
|-----------|------------------|-------------------|----------------------|
| 2026.7.6 | 70d6258 | 路线、环境与验收契约 | 今日按计划完成任务，颗粒度已对齐 |
| 2026.7.7 | 401c9e3..83caefb | 驱动/探针与目标 Linux | 能够独立分析实验问题及时修正操作中的错误 |
| 2026.7.7 | 4d19398..31b1ddb | 线程模型、协议测试与 GUI 证据 | 实验过程认真细致数据记录完整结论也较合理 |
| 2026.7.8 | b0fb0b6..6927be1 | 全功能验收与跨机演示 | 主动查阅相关资料进一步加深了对原理的理解 |
| 2026.7.8 | abad494..cf2a6c2 | 00-08 正式证据与截图 | 能够与组员有效配合并按时完成各项分配任务 |
| 2026.7.9 | eda12b6..6c51e5d | poll、三轮失败修复与最终通过 | 遇到问题保持耐心反复调试并记录经验与教训 |
| 2026.7.10 | 2ea2346..04f0b07 | 文档统一与目标回归关闭 | 本次实验思路清晰步骤合理效率较高 |
| 2026.7.11 | 略 | 正文内容、图表与证据整理 | 通过连续实践操作能力和理论水平都有所提升 |

第 9 章 总结与个人心得

9.1 项目总结

V-LED 从一个字符设备实验扩展为可验证的三端系统。核心价值不在功能数量，而在语义闭环：一页共享状态与每文件上下文职责清晰；错误码、边界、`version` 和快照可执行；并发和事件通知保持同一不变量；用户态与 GUI 有严格职责边界；目标 Linux 证据能够回溯到提交和原始日志。

最终结果证明驱动能在目标 6.17 内核零警告构建并动态装卸，业务命令、边界、多 FD、并发和 JSON 通过自动验收，Linux 到 Windows 的 UDP 链路有正式截图，`poll/wait queue` 扩展在不破坏 P1-P4 的前提下通过回归。更重要的是，项目保留了编译失败、栈帧警告和三次 P5 适配失败，展示了从现象到证据再到修复的真实工程过程。

9.2 乔思齐个人心得

本次课程设计让我最深的体会是，驱动程序的“能运行”和“语义正确”之间存在很大距离。开始设计字符设备时，`open`、`read`、`write`、`release` 看起来只是把几个回调填入 `file_operations`，但真正面对 `PAGE_SIZE`、多进程、分段读取和失败路径后，每个返回值都会影响用户态的判断。尤其是独立偏移问题，如果只在结构体中增加 `write_offset` 却没有让它参与容量和数据落点，形式上满足了要求，实际上无法被观察和验证。因此我把共享设备状态与每次 `open` 的文件上下文分开，明确哪些数据受设备 `mutex` 保护，哪些进度属于当前打开文件描述，并通过双 FD 探针验证。

目标 Linux 构建也改变了我对兼容性的认识。Windows 上的静态检查无法发现 `current` 宏冲突、`no_llseek` 接口变化和内核栈帧警告。第一次功能通过后，我本来容易把任务视为完成，但 1136 字节栈帧警告说明正确性还包含资源约束。把候选状态移到堆上并重新跑全部探针，让我理解了“零警告门禁”的意义。P5 中引入 `wait queue` 和 `poll` 时，我又认识到扩展功能不能改变已经冻结的快照、偏移和版本语义。三次失败最终定位为旧测试 `helper` 跨越了长期 FD 的版本边界，而不是简单修改驱动去迎合测试。

作为总体方案和驱动负责人，我还学习了如何用接口契约协调团队。驱动只生成完整 JSON，`bridge` 负责传输，Windows 再独立校验；任何一端都不能依赖未说明的隐含行为。通过最小提交、明确哈希、失败证据保留和 `fast-forward` 同步，问题可以被复现而不是依靠口头描述。今后面对更复杂的内核模块，我会继续坚持先定义不变量和失败语义，再写实现与测试，同时关注权限、资源释放、日志真实性和适用边界。

9.3 曹宇声个人心得

在用户态工具和 UDP bridge 的设计中，我认识到一个可靠工具最重要的不是输出好看的提示，而是准确反映系统调用结果。vled_cli 如果忽略短写、额外参数或 errno，就会把驱动错误包装成成功；自动验收如果只搜索 PASS 字符串，也无法证明设备状态真实变化。因此 CLI、probe 和 verify 都把退出码作为接口，写命令后重新读取 JSON、比较 version 和字段，失败立即停止并保留上下文。P3 首次运行把零长度 write 错当成空白命令，让我看到测试契约本身也可能错误。修复不是改变正确驱动，而是用三个空格构造真正的非法命令，并继续保留零长度返回 0 的冻结语义。

bridge 从周期读取改为 poll 事件驱动，是我对操作系统机制理解最具体的一次提升。周期轮询实现简单，但会重复唤醒和发送旧状态；长期打开设备后，poll、阻塞 read、O_NONBLOCK 和信号退出必须形成一致流程。P5 的几次失败尤其有价值：探针读完当前快照后再次 read 会按新语义阻塞；旧 helper 多读一次还可能把两个合法 JSON 拼接成一个非法结果。通过 strace、剩余长度和“首次完整协议校验即返回”的修改，我学会了从系统调用级证据判断问题究竟在驱动还是测试端。

网络联调还让我理解 UDP 的证据边界。sendto 返回成功并不等于 Windows 已经收到，所以正式结论必须同时依赖 Linux 日志、Windows GUI 和报文字段一致性。参数校验、SIGTERM 清理、设备不可用诊断和无监听边界同样属于工程质量。以后编写系统工具时，我会把可诊断性、可自动化、权限最小化和失败清理放在功能同等重要的位置，并避免用无限重试或静默默认值掩盖配置错误。

9.4 沈徐鹏个人心得

Windows 模拟器让我体会到图形界面并不是把网络数据直接写到控件上那么简单。Tkinter 有自己的事件循环，后台接收线程如果直接操作 Canvas、Label 或 Text，可能在高频报文、关闭窗口或不同解释器环境中产生跨线程异常。为此我把协议解析、UDP 接收、显示模型和 GUI 分成四层：接收线程只处理 socket、UTF-8、JSON 和不可变事件；有界 Queue 负责线程间传递；Tk 主线程通过 root.after 统一消费并完整替换状态。这个结构使协议测试可以不启动 GUI，也让非法报文不会出现“文字已改但颜色未改”的部分更新。

协议校验中的细节也让我印象深刻。字段存在并不代表类型正确，例如 Python 的 bool 是 int 子类，如果只用 isinstance(value, int)，true 可能被当作合法亮度或版本。实现最终要求普通整数、限制尺寸乘积、UTF-8 字节长度、RGB 三通道、mode 和报文总长，并把解析结果放入 frozen dataclass。手动颜色和亮度覆盖也不能覆盖后台最新 UDP 状态，否则恢复时无法立即显示最近数据；本地清屏必须明确只是预览行为，不能让用户误以为已经向 Linux 写入 CLEAR。

人工 GUI 验收让我认识到自动测试和视觉检查各有边界。无 GUI 测试可以稳定覆盖非法 JSON、socket 关闭、队列上限和模型规则，但滚动效果、界面冻结、日志可读性和恢复按钮仍需要人工观

察。连续 300 个合法报文、240 个非法报文和关闭后的端口检查，使“线程安全”变成可观察结果。今后开发 GUI 网络程序时，我会优先设计无界面可测的纯模块，让界面只负责展示与交互，并在关闭路径、资源上限和错误提示上投入足够精力。

参考文献

- [1] Linux Kernel Documentation. Overview of the Linux Virtual File System.
<https://docs.kernel.org/6.17/filesystems/vfs.html>.
- [2] Linux Kernel Documentation. Driver Basics. <https://docs.kernel.org/6.17/driver-api/basics.html>.
- [3] Linux Kernel Documentation. Device drivers infrastructure. <https://docs.kernel.org/6.17/driver-api/infrastructure.html>.
- [4] Linux Kernel Documentation. The Linux Kernel API. <https://docs.kernel.org/6.17/core-api/kernel-api.html>.
- [5] Linux man-pages project. open(2), read(2), write(2), poll(2). <https://www.kernel.org/doc/man-pages/>.
- [6] Python Software Foundation. queue - A synchronized queue class.
<https://docs.python.org/3/library/queue.html>.
- [7] Python Software Foundation. tkinter - Python interface to Tcl/Tk (Threading model).
<https://docs.python.org/3/library/tkinter.html>.
- [8] Python Software Foundation. threading - Thread-based parallelism.
<https://docs.python.org/3/library/threading.html>.
- [9] V-LED repository. README, PLAN, driver/tools/simulator documentation and source at commit 04f0b07. <https://github.com/qiaosiqi/V-LED>.
- [10] V-LED acceptance evidence. docs/ACCEPTANCE_MATRIX.md and docs/evidence/*/RESULTS.md, 2026-07-12。。相关内容见表 错误!未找到引用源。。相关内容见表 错误!未找到引用源。。